Architecture and Deployment Rationale

Project: RoomRadar QC

Purpose

This document explains the deployment architecture used for RoomRadar QC. It clari□es how the frontend, backend, and database communicate, why different hosting platforms were used, what the local development server means, and what security/networking controls are available in the current setup.

This document addresses the deployment-related feedback about the project architecture, Vercel frontend hosting, backend hosting, database hosting, and frontend server configuration.

---

1. Confirmed Deployment Architecture

The current RoomRadar QC deployment architecture is:

User Browser
 □
Vercel React/Vite Frontend
 □
Render Flask Backend API
 □
EC2 MySQL Database

In simpler terms:

Layer	Technology / Platform	Purpose
User Browser	Chrome, Safari, etc.	User interacts with the RoomRadar QC web app.
Frontend Hosting	Vercel	Hosts the built React/Vite static frontend.
Backend	Render	Hosts the Flask API server.

Hosting		
Database Hosting	EC2 MySQL	Stores semester, room, and schedule data.
Authentication	Firebase Authentication / Firebase Admin SDK	Handles admin login and verifies admin upload requests.

The original assumption was:

User Browser → Vercel Static React App → EC2 Flask API → EC2 MySQL

However, the current deployed version uses Render for the Flask backend API:

User Browser → Vercel Static React App → Render Flask API → EC2 MySQL

The MySQL database remains hosted on an EC2 instance.

---

2. Frontend Deployment: Vercel

Why Vercel Was Used

Vercel was used for the frontend because the frontend is a static React/Vite application after it is built. Vercel is designed to host static frontend applications and automatically handles the build and deployment process.

The main benefits of using Vercel are:

Benefit	Explanation
Simple frontend deployment	Vercel automatically builds and deploys the React/Vite frontend from GitHub.
HTTPS support	Vercel provides HTTPS for the deployed frontend URL.
Static file hosting	React/Vite builds into static HTML, CSS, and JavaScript files, which Vercel serves efficiently.
Easy redeployment	Pushing changes to GitHub can trigger a new frontend deployment.
Environment variables	Vercel supports frontend environment variables such as

	VITE_API_BASE_URL.

The production frontend is available through Vercel at:

https://csci-370-final-project.vercel.app

---

3. Why the Frontend Was Not Hosted on the Same Machine as the Backend

The frontend and backend were separated because they serve different purposes.

The frontend is a static React/Vite application. It does not need a Python runtime, database connection, or Flask server. It only needs to serve static files to the browser.

The backend is a Flask API. It requires:

- Python runtime
- Flask dependencies
- Firebase Admin SDK
- Environment variables
- Database connection to MySQL
- API route handling

Keeping them separate made the deployment process easier to manage:

Component	Hosting Choice	Reason
Frontend	Vercel	Best suited for static React/Vite deployment.
Backend	Render	Best suited for running a Python Flask API.
Database	EC2 MySQL	Existing database was already configured there.

An alternative design would be to host both the frontend and backend on the same EC2 instance. That would reduce the number of cloud providers, but it would require manually configuring a web server, static file hosting, HTTPS, routing, and backend process management.

For this project, Vercel reduced the overhead of managing the frontend hosting layer, while Render provided a simple way to run the Flask backend.

---

4. Local Development vs Production Deployment

The deployment guide mentions a frontend server running on port 5173. This is only for local development.

Local Development

During local development, Vite runs a development server, usually at:

http://localhost:5173

Sometimes, if port 5173 is already in use, Vite may run on:

http://localhost:5174

This local server is used only while developing and testing the React app on a local machine.

Production Deployment

In production, Vercel does not use the local Vite development server. Instead, Vercel builds the frontend using:

npm run build

Then Vercel serves the generated static files from the build output.

Therefore:

localhost:5173 is local development only.

It is not an Apache server, Tomcat server, or production frontend web server.

---

5. Backend Deployment: Render Flask API

The backend is a Flask application deployed on Render. The backend exposes API endpoints used by the frontend.

Examples:

GET /api/rooms/search
GET /api/rooms/:id/schedule
POST /api/admin/semester

The deployed backend URL is:

https://roomradar-backend-hfv7.onrender.com

The frontend uses an environment variable to know where the backend is located:

VITE_API_BASE_URL=https://roomradar-backend-hfv7.onrender.com

During local development, the frontend uses:

VITE_API_BASE_URL=http://127.0.0.1:5000

This allows the same frontend code to work in both local development and production.

---

6. Database Deployment: EC2 MySQL

RoomRadar QC uses a MySQL database hosted on an EC2 instance.

The database stores:

Table	Purpose
semesters	Stores uploaded semester names.
rooms	Stores room codes, building codes, and room capacity.
schedules	Stores scheduled class meetings connected to rooms and semesters.

The backend connects to the database using environment variables:

DB_HOST=...
DB_USER=...
DB_PASSWORD=...
DB_NAME=...
DB_PORT=3306

These values are not hardcoded into the source code. They are configured locally using a .env file and configured in Render through Render environment variables.

---

7. Frontend-to-Backend Communication

The frontend communicates with the backend using HTTP requests.

Example room search request:

https://roomradar-backend-hfv7.onrender.com/api/rooms/search?day=Monday&starttime=12:30&endtime=15:30

Example room schedule request:

https://roomradar-backend-hfv7.onrender.com/api/rooms/41/schedule?day=Monday

Example admin upload request:

https://roomradar-backend-hfv7.onrender.com/api/admin/semester

The frontend does not directly connect to the MySQL database. All database access goes through the Flask backend API.

---

8. CORS Configuration

Because the frontend and backend are hosted on different origins, Cross-Origin Resource Sharing, also known as CORS, must be configured on the Flask backend.

The frontend origin is:

https://csci-370-final-project.vercel.app

The backend origin is:

https://roomradar-backend-hfv7.onrender.com

Since these are different origins, the backend must allow requests from the frontend.

The backend CORS configuration allows expected origins such as:

http://localhost:5173
http://localhost:5174
https://csci-370-final-project.vercel.app

This allows both local development and deployed frontend requests to reach the backend API.

---

9. Security and Networking Controls on Vercel

Vercel hosts the frontend as static files. Because of this, the team has different security controls than it would have on a self-managed EC2 web server.

Controls Available on Vercel

Control	Explanation
HTTPS	Vercel provides HTTPS for deployed frontend URLs.
Environment variables	Vercel allows the frontend to store deployment-specific values like the

	backend API base URL.
GitHub-based deployment	Deployments can be tied to the GitHub repository and branch.
Build settings	The team can control build commands, output directory, and environment configuration.
Domain settings	The team can configure custom domains if needed.
Frontend code security	The team controls frontend routing, form validation, and API request behavior.

Controls Not Directly Available on Vercel

Limitation	Explanation
Low-level firewall rules	The team cannot configure Vercel like an EC2 security group with custom source/destination IP rules.
Specific inbound ports	The team does not manually open or close frontend ports like 80, 443, or 5173 in production.
Internal CDN networking	Vercel manages its own CDN/network infrastructure.
Apache/Tomcat configuration	The frontend is not hosted through Apache, Tomcat, or a Java servlet container.
web.xml style access rules	Since this is not a Java servlet app, access rules are handled through frontend routing and backend API protection instead.

For this project, sensitive operations are not protected by the frontend alone. Admin-only actions are protected by the backend API using Firebase token verification.

---

10. Security and Networking Controls on the Backend

The backend has more control over application security because it handles API requests, authentication checks, and database access.

Backend security controls include:

Control	Explanation
Firebase token verification	Admin upload requests require a valid Firebase bearer token.
CORS configuration	The backend controls which frontend origins can access the API.
Environment variables	Database credentials and deployment-specific values are stored outside the source code.
Database access control	The backend controls how the frontend can access data. The frontend does not connect directly to MySQL.
API route validation	Backend routes validate request parameters before processing.
SQL parameterization	Backend queries use parameterized SQL values instead of directly concatenating raw user input into SQL strings.

The backend is the main enforcement point for protected actions. The frontend can hide or show UI elements, but real protection must happen on the backend.

---

11. Cybersecurity Guardrails in the Current Design

RoomRadar QC uses several guardrails in the current deployment design:

Area	Guardrail
Admin authentication	Firebase login is required before admin upload.
Bearer token validation	The backend verifies the Firebase ID token before processing CSV uploads.
Secret handling	Database credentials are stored in environment variables, not hardcoded in source code.
Firebase credentials	Firebase service account credentials are excluded from GitHub.
CORS	Backend allows frontend requests only from expected frontend development and deployment origins.
Database isolation	The frontend cannot directly access MySQL.

HTTPS	Vercel and Render provide HTTPS URLs for frontend and backend communication.

---

12. Cost and Complexity Consideration

The architecture uses multiple services:

Vercel for frontend
Render for backend
EC2 for MySQL database
Firebase for authentication

For a production system, this architecture may be more complex than necessary. However, for this class project, each service was used to solve a specific development or deployment problem:

Service	Reason Used
Vercel	Simple React/Vite frontend hosting.
Render	Simple Python Flask backend hosting.
EC2 MySQL	Existing configured MySQL database environment.
Firebase	Admin authentication and token verification.

A simpler architecture could host the frontend, backend, and database closer together on one cloud provider. For example, the team could host the Flask backend and static frontend on the same EC2 server and use the same EC2-hosted MySQL database.

That would reduce provider complexity but would require more manual setup, including web server configuration, HTTPS setup, process management, and static file serving.

For the current project, the separated architecture allowed the team to focus more on implementing the system features and less on manually configuring a full production server environment.

---

13. Alternative Architecture Considered

An alternative architecture would be:

User Browser □ EC2 Web Server □ Flask Backend □ EC2 MySQL

In this design, EC2 could host both:

- The built React static frontend files
- The Flask backend API
- The MySQL database

Potential advantages:

- Fewer hosting providers
- More direct control over firewall rules
- More control over server configuration
- Backend and database are on the same cloud provider

Potential disadvantages:

- More manual server administration
- Need to configure static file hosting
- Need to configure HTTPS
- Need to configure process management for Flask
- Need to handle web server setup manually
- Higher risk of deployment misconfiguration for a short class project

The current design uses managed deployment platforms to reduce setup time.

---

14. Summary

RoomRadar QC uses the following production architecture:

User Browser → Vercel React/Vite Frontend → Render Flask Backend API → EC2 MySQL Database

Vercel is used for the frontend because React/Vite builds into static files that Vercel can host easily over HTTPS. Render is used for the backend because the Flask API requires a Python runtime and environment variables. EC2 MySQL is used as the database layer.

The localhost:5173 frontend server mentioned in the deployment guide is only the Vite development server used during local development. It is not the production frontend server.

The team can control application-level security, environment variables, frontend routing, API URLs, backend CORS, backend authentication checks, and database access logic. The team cannot directly control Vercel's internal CDN networking, source IPs, low-level firewall rules, or Apache/Tomcat-style server configuration because Vercel is a managed frontend hosting platform.